NLP Interview Questions and Answers

1. What is Natural Language Processing (NLP) and what are its main applications?

Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) that enables computers to understand, process and generate human language.

It helps computers work with text and speech.

Main applications:

1. Chatbots – ChatGPT and customer support bots.
2. Machine Translation – Google Translate.
3. Sentiment Analysis – Identifying positive or negative reviews.
4. Speech Recognition – Converting speech into text.
5. Text Summarisation – Creating short summaries.
6. Spam Detection – Identifying unwanted emails.

Interview answer:

"NLP is a branch of AI that helps computers understand, process and generate human language. Its applications include chatbots, machine translation, sentiment analysis, speech recognition and text summarisation."

2. What is the difference between NLP, NLU and NLG?

These three concepts have different roles:

- NLP: Processes human language.
- NLU (Natural Language Understanding): Understands the meaning and intention of language.
- NLG (Natural Language Generation): Generates human-like language.

Concept	Main purpose	Example

NLP	Processes language	Processing a sentence
NLU	Understands meaning	Understanding "Book a ticket"
NLG	Generates language	"Your ticket is booked"

Interview answer:

"NLP is the overall technology for processing human language. NLU focuses on understanding the meaning and intention, whereas NLG focuses on generating human-readable responses."

3. How does NLP differ from traditional text processing?

Traditional text processing mainly works with words, patterns and keywords. NLP goes further by analysing meaning, context and language structure.

Example:

Sentence: "This movie is amazing!"

- Traditional processing: Identifies words such as "movie" and "amazing".
- NLP: Identifies the sentence as expressing positive sentiment.

Interview answer:

"Traditional text processing mainly focuses on keywords and patterns, whereas NLP tries to understand the meaning, context and intention of human language."

4. What are the major challenges in processing human language?

Human language is complex because the same word or sentence can have different meanings depending on the context.

Major challenges:

1. Ambiguity – A word can have multiple meanings.
2. Context – Understanding the situation behind a sentence.

3. Sarcasm – Understanding hidden or opposite meanings.
4. Slang – Understanding informal language.
5. Grammar – Handling different sentence structures.
6. Multilingualism – Processing different languages.

Example:

"The bank is near the river."

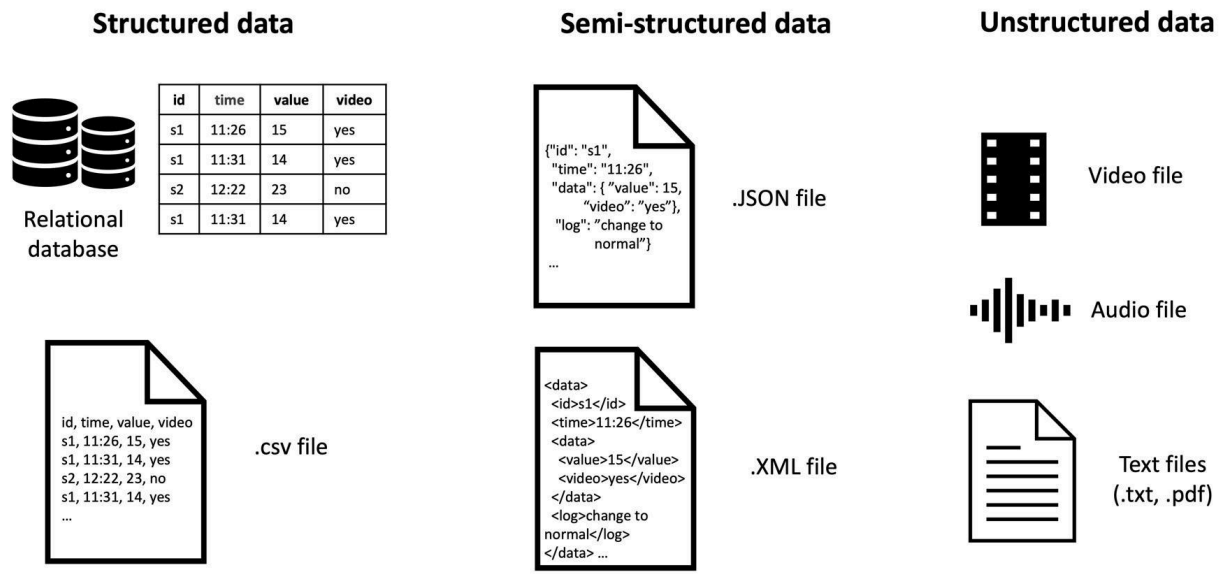
Here, bank means the land beside a river, not a financial institution.

Interview answer:

"The major challenges in NLP include ambiguity, understanding context, sarcasm, slang, grammatical variations and processing multiple languages."

5. What is the difference between structured, semi-structured and unstructured data?

Data can be classified into three types based on how it is organised.



Type	Explanation	Examples
Structured	Organised in rows and columns	Excel, SQL tables

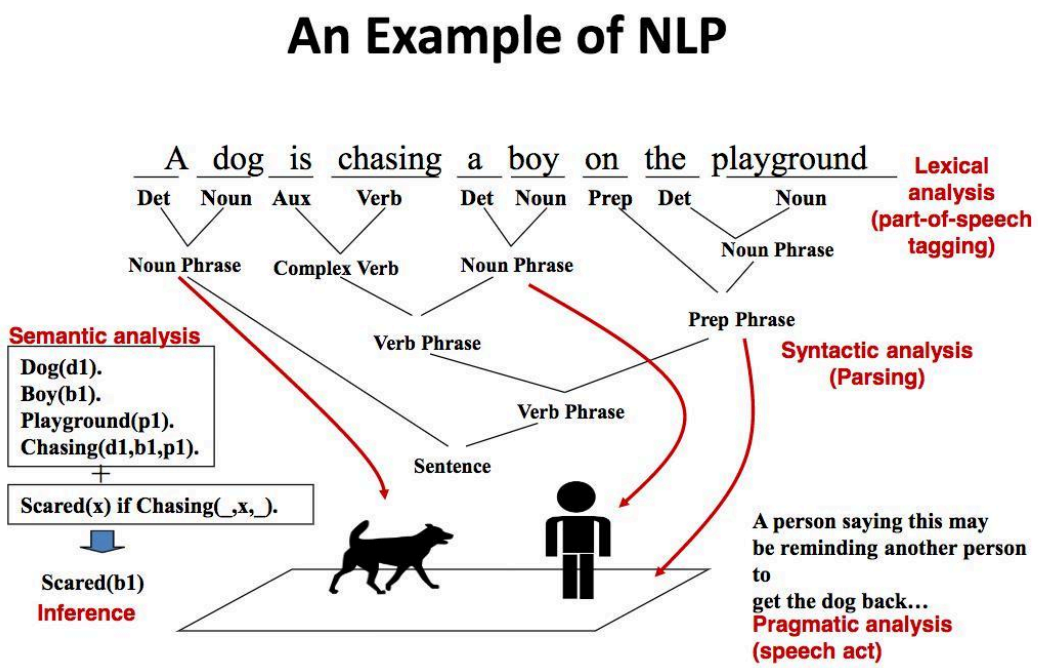
Semi-structured	Has some organisation but no fixed table structure	JSON, XML
Unstructured	Does not follow a predefined structure	Images, videos, audio, text documents

Interview answer:

"Structured data follows a fixed format, such as database tables. Semi-structured data has some organisation, such as JSON and XML. Unstructured data has no predefined format, such as images, videos and text documents."

6. What are lexical, syntactic, semantic and pragmatic analysis?

These are different levels of analysing human language.



Analysis	Meaning	Example

Lexical	Analysing words and their forms	Identifying "running" as a word
Syntactic	Analysing sentence grammar	"She reads a book"
Semantic	Understanding the literal meaning	Understanding what the sentence says
Pragmatic	Understanding meaning using context and intention	"Can you open the door?" is a request

Interview answer:

"Lexical analysis deals with words, syntactic analysis deals with grammar, semantic analysis deals with sentence meaning, and pragmatic analysis understands meaning based on context and intention."

7. What is ambiguity in natural language? Explain lexical and syntactic ambiguity with examples.

Ambiguity occurs when a word or sentence has more than one possible meaning.

There are two common types:

1. Lexical ambiguity

A single word has multiple meanings.

Example:

"She went to the bank."

- Bank = Financial institution.
- Bank = Riverbank.

2. Syntactic ambiguity

A sentence has multiple possible meanings because of its grammatical structure.

Example:

"I saw the man with a telescope."

- I used a telescope to see the man.
- I saw a man who had a telescope.

Interview answer:

"Ambiguity means a word or sentence can have multiple meanings. Lexical ambiguity occurs when a word has multiple meanings, whereas syntactic ambiguity occurs when a sentence has multiple interpretations because of its grammatical structure."

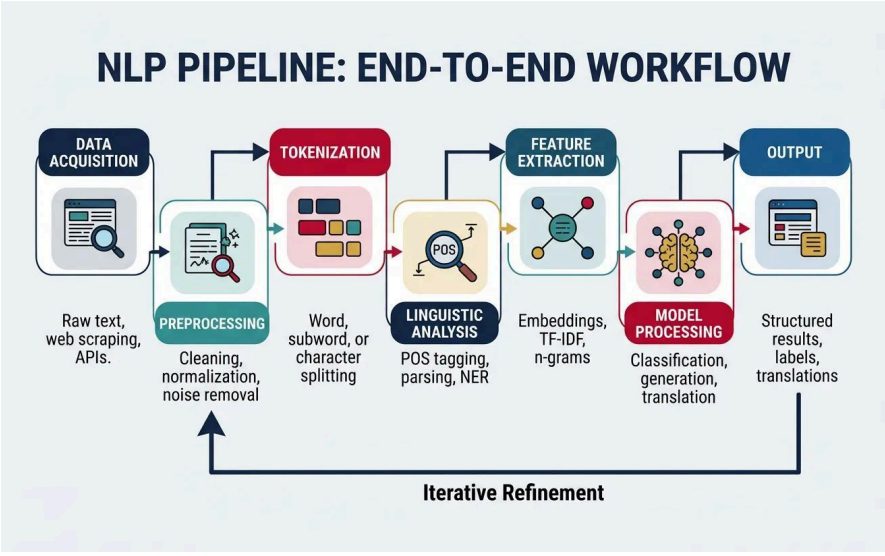
8. What is a corpus in NLP, and how is it used to develop language models?

A corpus is a large collection of text or language data used to train and evaluate NLP models.

Examples:

- Wikipedia articles.
- News articles.
- Books.
- Customer reviews.
- Social media posts.

How is it used?



1. Collect text data.

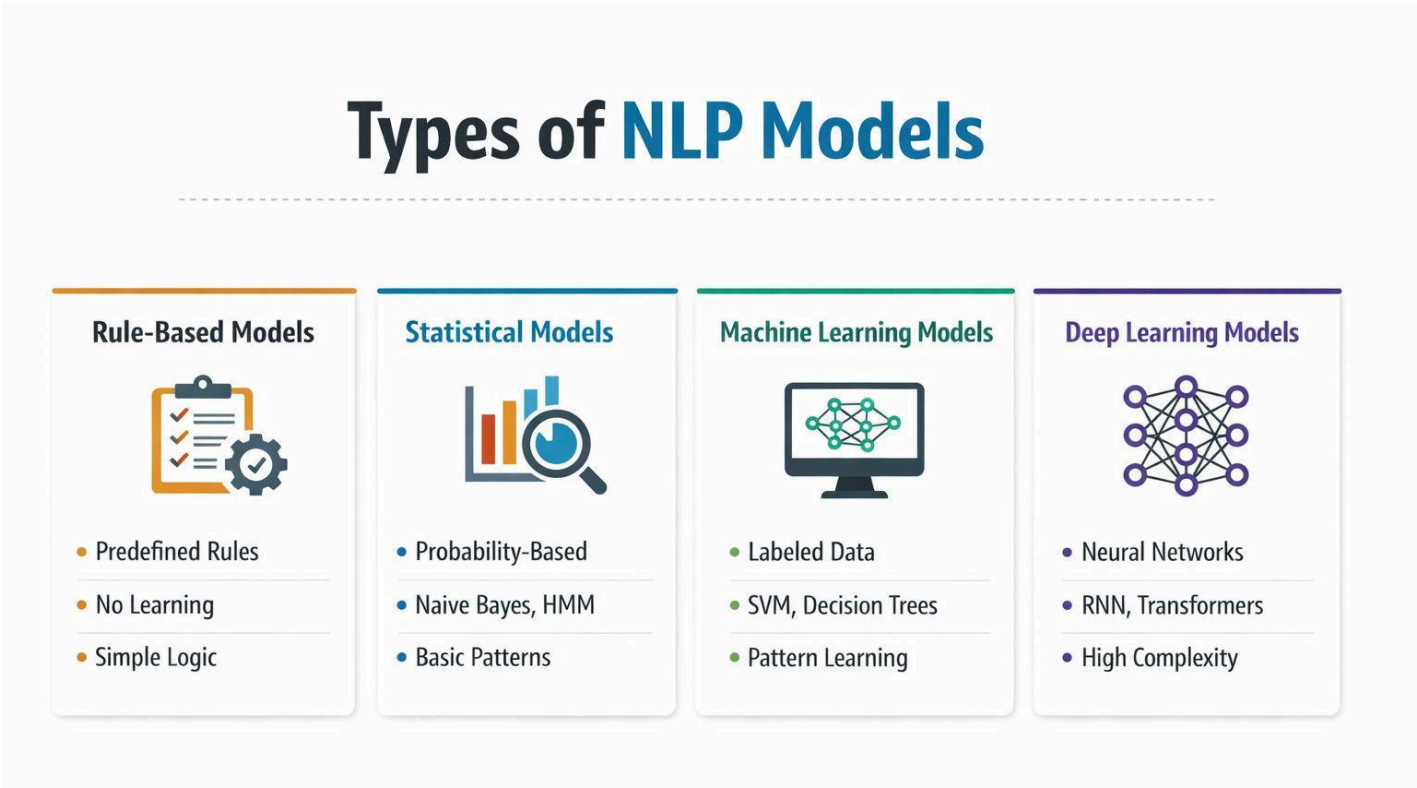
2. Clean and preprocess the data.
3. Convert text into tokens.
4. Train the language model.
5. Evaluate the model.

Interview answer:

"A corpus is a collection of text data used to train and evaluate NLP models. For example, Wikipedia articles and news articles can be used as training data to help a language model learn language patterns."

9. What is the difference between rule-based NLP, statistical NLP and neural NLP?

These are three approaches used to build NLP systems.



Approach	How it works	Example

Rule-based NLP	Uses predefined rules	Grammar checking using rules
Statistical NLP	Uses probability and statistical methods	Naive Bayes text classification
Neural NLP	Uses neural networks and deep learning	Transformers, BERT, GPT

Example: Spam detection

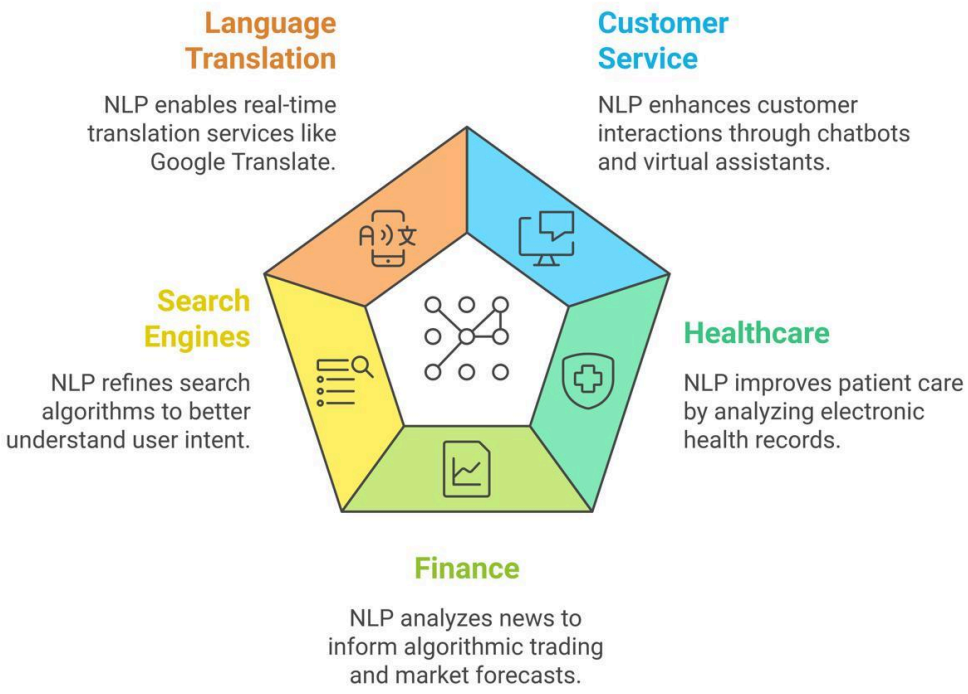
- Rule-based: If an email contains certain suspicious words, mark it as spam.
- Statistical: Calculate the probability that an email is spam based on learned patterns.
- Neural: Use a neural network to learn complex patterns from email data.

Interview answer:

"Rule-based NLP uses predefined linguistic rules. Statistical NLP uses probability and statistical techniques to learn from data. Neural NLP uses neural networks and deep learning to understand complex language patterns."

10. What are the common real-world applications of NLP in healthcare, finance, education and customer service?

NLP is used in different industries to process text, understand information and automate communication.



Industry	NLP applications	Example
Healthcare	Medical report analysis, clinical documentation	Extracting information from medical reports
Finance	Fraud detection, sentiment analysis	Analysing financial news
Education	Automated grading, learning assistants	AI tutoring chatbot
Customer Service	Chatbots, sentiment analysis	Automatic customer support

Interview answer:

"In healthcare, NLP helps analyse medical reports. In finance, it supports fraud detection and financial sentiment analysis. In education, it supports automated grading and AI tutors. In customer service, it powers chatbots and automated support systems."

11. What is text preprocessing, and why is it important in NLP?

Text preprocessing is the process of cleaning and preparing raw text before giving it to an NLP model.

Raw text may contain unnecessary spaces, punctuation, spelling mistakes, URLs and unwanted words.

Common preprocessing steps:

1. Lowercasing – Convert text into lowercase.
2. Tokenisation – Split text into smaller units.
3. Stop-word removal – Remove selected common words.
4. Stemming – Reduce words to their root forms.
5. Lemmatization – Convert words into their dictionary forms.
6. Removing unwanted characters – Clean unnecessary symbols.

Example:

Original text:
`"HELLO!!! I am Learning NLP Concepts."`

After preprocessing:
`hello learning nlp concepts`

Interview answer:

"Text preprocessing is the process of cleaning and preparing raw text before using it in an NLP model. It includes tokenisation, lowercasing, stop-word removal, stemming and lemmatisation. It helps improve the quality and consistency of text data."

12. What is text normalisation? Give examples of normalising informal text.

Text normalisation is the process of converting different forms of text into a standard and consistent format.

It is especially useful when processing informal text from social media and messaging applications.

Examples:

Original text	Normalised text
u	you
gonna	going to
wanna	want to
gr8	great
pls	please
luv	love
sooo good	so good

Example:

Original:
`"Heyyy, u r gr8!!!"`

Normalised:
`"Hey, you are great!"`

Interview answer:

"Text normalisation converts informal or inconsistent text into a standard format. For example, 'u' becomes 'you', 'gr8' becomes 'great', and 'gonna' becomes 'going to'. It helps NLP models process text more consistently."

13. What is tokenisation, and what are the different types of tokenisation?

Tokenisation is the process of splitting text into smaller units called tokens.

A token can be a sentence, word, subword or character.

Different types:

1. Sentence tokenisation – Splits text into sentences.
2. Word tokenisation – Splits sentences into words.
3. Subword tokenisation – Splits words into smaller meaningful or frequent units.
4. Character tokenisation – Splits text into individual characters.

Example:

Input:

```
"I love Natural Language Processing."
```

Word tokens:

```
["I", "love", "Natural", "Language", "Processing"]
```

Interview answer:

"Tokenisation is the process of splitting text into smaller units called tokens. Common types include sentence, word, subword and character tokenisation. It is an important first step in many NLP tasks."

14. What is the difference between sentence tokenisation and word tokenisation?

Both are tokenisation techniques, but they split text at different levels.

Example:

Input:

```
"I love Python. I am learning NLP."
```

1. Sentence tokenisation:

Splits the text into individual sentences.

```
None
[    "I love Python.",    "I am learning NLP."]
```

2. Word tokenisation:

Splits the text into individual words.

```
None
[    "I", "love", "Python",    "I", "am", "learning", "NLP"]
```

Interview answer:

"Sentence tokenisation divides text into individual sentences, whereas word tokenisation divides sentences into individual words or tokens."

15. What is the difference between stemming and lemmatisation?

Both techniques reduce words to a base form, but they work differently.

1. Stemming

Stemming removes prefixes or suffixes using simple rules. The result may not always be a meaningful dictionary word.

Examples:

- Playing → Play
- Studies → Studi
- Connected → Connect

2. Lemmatisation

Lemmatisation converts words into their correct dictionary form (lemma), considering vocabulary and sometimes grammar.

Examples:

- Running → Run

- Better → Good
- Studies → Study
- Was → Be

Python example:

```
None
from nltk.stem import PorterStemmer, WordNetLemmatizerstemmer = PorterStemmer()lemmatizer = WordNetLemmatizer()print(stemmer.stem("studies"))# studiprint(lemmatizer.lemmatize("studies"))# study
```

Interview answer:

"Stemming removes word endings to produce a root-like form, whereas lemmatisation converts words into their correct dictionary form. Stemming is faster, but lemmatisation generally produces more meaningful results."

16. What are stop words, and when should they be removed?

Stop words are common words that may carry little useful information for certain NLP tasks.

Examples:

- is
- the
- a
- an
- and
- are
- of

Example:

Original sentence:

```
"I am learning the basics of NLP."
```

After stop-word removal:

```
"learning basics NLP"
```

When should we remove them?

- Search engines.
- Simple text classification.
- Keyword extraction.
- Some traditional machine learning tasks.

When should we keep them?

- Sentiment analysis.
- Machine translation.
- Question answering.
- Language models.

For example, removing "not" from "This is not good" changes the meaning.

Interview answer:

"Stop words are frequently occurring words such as 'is', 'the' and 'and'. We remove them when they do not contribute useful information to a task. However, we should keep them when they affect sentence meaning or context."

17. What is case folding, and when might converting text to lowercase be inappropriate?

Case folding is the process of converting text into a consistent letter case, usually lowercase.

Example:

Original:

```
"NLP IS INTERESTING"
```

After case folding:

```
"nlp is interesting"
```

Why is it used?

It helps treat words with different capitalisation as the same word.

For example:

- Apple
- APPLE

- apple

When should we avoid it?

1. Named entities – US and us have different meanings.
2. Acronyms – AI, NLP and NASA.
3. Case-sensitive programming languages.
4. Tasks where capitalisation provides useful information.

Interview answer:

"Case folding converts text into a consistent case, usually lowercase. It reduces unnecessary differences between words. However, it may be inappropriate when capitalisation carries important information, such as names, acronyms or case-sensitive text."

18. How would you handle punctuation, numbers, URLs and special characters during preprocessing?

During text preprocessing, we handle unnecessary punctuation, numbers, URLs and special characters depending on the NLP task.

Element	Handling	Example
Punctuation	Remove or retain	Hello!!! → Hello
Numbers	Remove or normalise	123 → NUM
URLs	Remove or replace	website.com → URL
Special characters	Remove or normalise	@user → user
Extra spaces	Remove	Multiple spaces → Single space

Example:

Original:

```
"Visit https://example.com!!! Price is $500 @store"
```

After preprocessing:

```
"Visit URL Price is NUM store"
```

Important: We should not blindly remove everything.

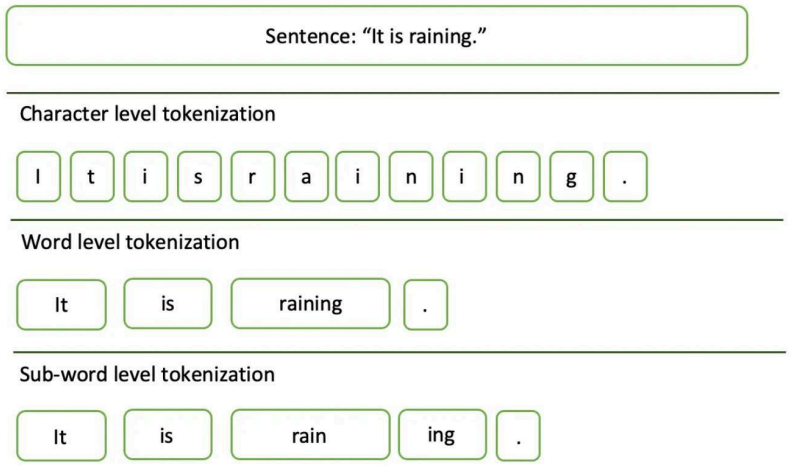
- Numbers are important in financial data.
- Punctuation is important in sentiment analysis.
- URLs may be important in spam detection.
- Special characters may carry meaning in social media text.

Interview answer:

"I handle punctuation, numbers, URLs and special characters based on the NLP task. I may remove unnecessary punctuation, replace URLs with a common token, normalise numbers and clean unwanted characters while preserving useful information."

19. What is the difference between whitespace tokenisation, subword tokenisation and character tokenisation?

These three techniques split text into tokens in different ways.



1. Whitespace tokenisation

Splits text based on spaces.

Example:

`"I love NLP"`

Output:

```
None
["I", "love", "NLP"]
```

2. Subword tokenisation

Splits words into smaller units or subword pieces.

Example:

`"playing"`

Possible output:

```
None
["play", "##ing"]
```

3. Character tokenisation

Splits text into individual characters.

Example:

`"NLP"`

Output:

```
None
["N", "L", "P"]
```

Type	Splitting method	Common use
Whitespace	Spaces	Simple text processing
Subword	Word pieces	BERT, GPT
Character	Individual characters	Character-level NLP models

Interview answer:

"Whitespace tokenisation splits text using spaces. Subword tokenisation divides words into smaller units, while character tokenisation splits text into individual characters. Subword tokenisation is widely used in modern language models such as BERT and GPT."

20. How would you preprocess noisy social media text containing emojis, hashtags, abbreviations and spelling mistakes?

Social media text often contains informal language, emojis, hashtags, abbreviations, spelling mistakes and URLs.

We need to clean and normalise the text while preserving its meaning.

Example:

Original text:

```
"OMG!!! I luv this movie 😍😍 #Amazing https://abc.com"
```

Preprocessing steps:

1. Lowercase the text.
2. Remove unnecessary punctuation.
3. Convert abbreviations into standard words.
4. Correct spelling mistakes where appropriate.

5. Convert emojis into meaningful descriptions or retain them.
6. Extract or process hashtags.
7. Remove or replace URLs.
8. Remove unnecessary spaces.

Example output:

```
"oh my god i love this movie smiling_face_with_heart_eyes amazing"
```

Python example:

```
None
import retext = "OMG!!! I luv this movie 😍 #Amazinghttps://abc.com

"# Convert to lowercase

text = text.lower()

# Expand abbreviations

text = text.replace("omg", "oh my god")

text = text.replace("luv", "love")

# Remove URLs

text = re.sub(r"https?://\S+", "", text)

# Remove hashtag symbol but keep the word

text = re.sub(r"#(\w+)", r"\1", text)

# Remove extra punctuation

text = re.sub(r"[!]+", "", text)

# Remove extra spaces

text = " ".join(text.split())print(text)
```

Output:

```
None
oh my god love this movie 😍 amazing
```

Interview answer:

"I would clean and normalise social media text by converting it to lowercase, expanding abbreviations, correcting spelling mistakes, handling emojis, extracting hashtags and removing unnecessary URLs and punctuation. I would preserve meaningful information because emojis and hashtags can be useful for sentiment analysis."

21. What are regular expressions, and how are they useful in NLP?

Regular Expressions (Regex) are patterns used to search, identify, extract, replace or remove specific text from a string.

In Python, we use the `re` module to work with regular expressions.

Common applications in NLP:

1. Extracting email addresses.
2. Extracting telephone numbers.
3. Finding hashtags and mentions.
4. Removing HTML tags.
5. Identifying URLs.
6. Cleaning unwanted characters.

Python example:

```
None
import retext = "Contact us at john@gmail.com"

result = re.findall(r"\S+@\S+", text)print(result)
```

Output:

```
None
['john@gmail.com']
```

Interview answer:

"Regular Expressions are patterns used to search, extract, replace and clean text. In NLP, they are useful for extracting emails, phone numbers, URLs, hashtags and removing unwanted characters."

22. What is the difference between `re.match()`, `re.search()` and `re.findall()` in Python?

All three functions are available in Python's `re` module, but they work differently.

Consider this text:

```
None
import retext = "Python is easy. I love Python."
```

1. `re.match()`

Checks for a pattern only at the beginning of the string.

```
None
print(re.match("Python", text))
```

Output: Returns a match object.

2. `re.search()`

Searches for the first occurrence of a pattern anywhere in the string.

```
None
print(re.search("love", text))
```

Output: Returns a match object.

3. `re.findall()`

Returns all matching occurrences as a list.

```
None
print(re.findall("Python", text))
```

Output:

```
None
['Python', 'Python']
```

Comparison:

Function	Purpose	Result
`re.match()`	Checks beginning	First match object
`re.search()`	Searches anywhere	First match object
`re.findall()`	Finds all matches	List of matches

Interview answer:

"`re.match()` checks for a match at the beginning of a string. `re.search()` finds the first match anywhere in the string, whereas `re.findall()` returns all matching occurrences as a list."

23. How would you extract email addresses and telephone numbers from text using regular expressions?

We can use regular expression patterns with Python's `re.findall()` function to extract emails and telephone numbers.

Python example:

```
None
import re
```

```
text = """
Contact: john@gmail.com
Phone: +91 9876543210
Email: student@yahoo.com
"""
# Extract email addresses
emails = re.findall(
    r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
    text
)
# Extract Indian mobile numbers
phones = re.findall(
    r"(?:\+91[- ]?)?[6-9]\d{9}",
    text
)
print("Emails:", emails)
print("Phone numbers:", phones)
```

Output:

```
None
Emails: ['john@gmail.com', 'student@yahoo.com']
```

```
Phone numbers: ['+91 9876543210']
```

Interview answer:

"I use regular expressions with `re.findall()` to extract email addresses and telephone numbers. I create suitable patterns for the required format and retrieve all matching values from the text."

Note: These patterns cover common formats, not every valid email or international telephone number. Real-world validation may require more advanced checks.

24. How would you remove HTML tags from a text document?

HTML tags are used to structure web pages. Sometimes we need to remove these tags and extract only the readable text.

For example:

Original HTML:

```
None
<h1>Hello World</h1>

<p>Welcome to NLP</p>
```

After removing HTML tags:

```
None
Hello World

Welcome to NLP
```

Python example:

```
None
import re
```

```
html_text = """
<h1>Hello World</h1>
<p>Welcome to NLP</p>
"""
clean_text = re.sub(r"<[^>]+>", "", html_text)
print(clean_text)
```

Output:

```
None
Hello World

Welcome to NLP
```

Interview answer:

"We can use regular expressions with `re.sub()` to remove simple HTML tags from text. For example, the pattern `<[^>]+>` matches HTML tags. For complex HTML documents, I would prefer an HTML parser such as BeautifulSoup because regular expressions cannot reliably handle all HTML structures."

25. How would you identify and extract hashtags, mentions and URLs from social media posts?

Social media posts contain special elements such as:

- Hashtags – Words starting with `#`.
- Mentions – Usernames starting with `@`.
- URLs – Website links starting with HTTP or HTTPS.

We can extract these elements using regular expressions.

Python example:

```
None
import re

text = """

Learning NLP with @john.

#Python #AI

Visit https://example.com

"""

# Extract hashtags

hashtags = re.findall(r"#\w+", text)

# Extract mentions

mentions = re.findall(r"@\w+", text)

# Extract URLs

urls = re.findall(r"https?://\S+", text)

print("Hashtags:", hashtags)

print("Mentions:", mentions)

print("URLs:", urls)
```

Output:

```
None
Hashtags: ['#Python', '#AI']

Mentions: ['@john']

URLs: ['https://example.com']
```

Interview answer:

"I use regular expressions to identify hashtags using `#`, mentions using `@` and URLs using HTTP or HTTPS patterns. Then I use `re.findall()` to extract all matching elements."

26. What is the difference between greedy and non-greedy matching in regular expressions?

Greedy and non-greedy matching determine how much text a regular expression captures.

1. Greedy matching

Greedy matching captures as much text as possible.

2. Non-greedy matching

Non-greedy matching captures as little text as possible while satisfying the pattern.

Example:

```
None
import retext = "<b>Hello</b><b>World</b>"# Greedy
matchingprint(re.findall(r"<b>.*</b>", text))# Non-greedy
matchingprint(re.findall(r"<b>.*?</b>", text))
```

Output:

```
None
Greedy:

['<b>Hello</b><b>World</b>']

Non-greedy:

['<b>Hello</b>', '<b>World</b>']
```

Important symbols:

Symbol	Meaning

`*`	Zero or more (greedy)
`*?`	Zero or more (non-greedy)
`+`	One or more (greedy)
`+?`	One or more (non-greedy)

Interview answer:

"Greedy matching captures the maximum possible text, whereas non-greedy matching captures the minimum possible text. We use a question mark after a quantifier, such as `*?`, to make it non-greedy."

27. How would you remove duplicate whitespace and standardise line breaks in text?

Sometimes text contains multiple spaces, tabs and inconsistent line breaks.

We can use regular expressions to clean and standardise them.

Python example:

```
None
import re

text = """

Hello     World

Welcome     to NLP

"""
```

```
# Replace multiple spaces and tabs with one space

text = re.sub(r"[ \t]+", " ", text)

# Standardise line breaks

text = re.sub(r"\r\n?|\n", "\n", text)

# Remove extra blank lines

text = re.sub(r"\n+", "\n", text)

print(text.strip())
```

Output:

```
None
Hello World

Welcome to NLP
```

Interview answer:

"I use `re.sub()` to replace multiple spaces and tabs with a single space. I also standardise different line-break formats and remove unnecessary blank lines to make the text consistent."

28. What are Unicode and UTF-8, and why do they matter in multilingual NLP?

Unicode is a universal character standard that assigns a unique code point to characters from different languages and symbol systems.

UTF-8 is a character encoding system that represents Unicode characters using one to four bytes.

For example, Unicode supports:

- English – Hello

- Malayalam – നമസ്കാരം
- Hindi – नमस्ते
- Chinese – 你好
- Emojis – 😊

Why are they important in NLP?

1. They allow computers to process multiple languages.
2. They support different scripts and symbols.
3. They help prevent text encoding problems.
4. They allow multilingual NLP applications to work correctly.

Python example:

```
None
text = "Hello നമസ്കാരം 😊"

# Encode into UTF-8 bytes

encoded = text.encode("utf-8")

# Decode back into text

decoded = encoded.decode("utf-8")

print(decoded)
```

Output:

```
None
Hello നമസ്കാരം 😊
```

Interview answer:

"Unicode is a standard that represents characters from different languages, while UTF-8 is an encoding format used to store and transmit Unicode text. They are important in multilingual NLP because they allow us to process different languages, scripts and symbols correctly."

29. How would you detect and correct common spelling errors in text?

Spelling correction identifies incorrectly written words and replaces them with their correct forms.

For example:

- `recieve` → `receive`
- `teh` → `the`
- `langauge` → `language`
- `pyhton` → `python`

Common approaches:

1. Dictionary-based correction – Compare words with a dictionary.
2. Edit distance – Measure how many character changes are needed to correct a word.
3. Fuzzy matching – Find words that are similar to the incorrect word.
4. Machine learning – Learn spelling patterns from training data.

Python example using TextBlob:

```
None
from textblob import TextBlob

text = "I love machne lerning"

blob = TextBlob(text)

corrected = blob.correct()

print(corrected)
```

Possible output:

```
None
I love machine learning
```

Interview answer:

"I can detect and correct spelling mistakes using dictionary-based methods, edit distance, fuzzy matching or machine learning. Python libraries such as TextBlob can also help with spelling correction."

Note: Automatic spelling correction is not always accurate, especially for names, technical terms and multilingual text.

30. What are the limitations of using regular expressions as the primary method for understanding natural language?

Regular expressions are useful for finding patterns and cleaning text, but they cannot fully understand human language.

Major limitations:

1. No real understanding: Regex cannot understand the actual meaning of a sentence.
2. Context limitations: It cannot reliably understand the meaning of a word based on context.
3. Ambiguity: It struggles with words that have multiple meanings.
4. Complex grammar: It is difficult to handle complex sentence structures using regex alone.
5. Language variations: Different languages have different grammatical rules and writing systems.

Example:

Consider:

- "I went to the bank to deposit money."
- "I sat near the bank of the river."

The word "bank" has different meanings in these sentences.

Regular expressions can identify the word "bank", but they cannot reliably understand its meaning from context.

Interview answer:

"Regular expressions are useful for text cleaning and pattern extraction, but they cannot understand the meaning, context, ambiguity or complex grammar of human language. Therefore, NLP techniques such as machine learning and deep learning are needed for deeper language understanding."

NLP Interview Questions and Answers (31–40): Morphology, POS Tagging and Parsing

These answers are designed for beginners preparing for NLP technical interviews. Each question includes a simple explanation, practical examples and a short interview-ready answer.

31. What is morphology, and how does it relate to NLP?

Morphology is the study of how words are formed and how they change their forms.

In NLP, morphology helps computers understand the structure of words, including prefixes, suffixes and root words.

Examples:

Word	Morphological form
Play	Root word
Playing	Play + ing
Played	Play + ed
Unhappy	Un + happy
Teachers	Teach + er + s

Why is it important in NLP?

- Helps identify root words.
- Supports stemming and lemmatisation.
- Helps understand different word forms.
- Improves language processing.

Interview answer:

"Morphology is the study of word structure and formation. In NLP, it helps identify root words, prefixes and suffixes, allowing computers to understand different forms of a word."

32. What are morphemes, and what is the difference between free and bound morphemes?

A morpheme is the smallest unit of a word that carries meaning or grammatical information.

There are two main types:

1. Free morphemes

These can exist independently as meaningful words.

Examples:

- Book
- Happy
- Play
- Teach

2. Bound morphemes

These cannot exist independently. They must be attached to another morpheme.

Examples:

- `un-` in unhappy.
- `-ing` in playing.
- `-ed` in played.
- `-s` in books.

Comparison:

Free morpheme	Bound morpheme
Can stand alone	Cannot stand alone
Book	-s

Happy	un-
Play	-ing

Interview answer:

"A morpheme is the smallest meaningful unit of a word. Free morphemes can exist independently, such as 'book', whereas bound morphemes must attach to another word, such as '-s' or '-ing'."

33. What is Part-of-Speech (POS) tagging?

Part-of-Speech tagging is the process of identifying the grammatical category of each word in a sentence.

It identifies whether a word is a noun, verb, adjective, adverb and so on.

Example:

Sentence:

```
"The smart student studies carefully."
```

Word	POS tag	Category
The	DET	Determiner
smart	ADJ	Adjective
student	NOUN	Noun
studies	VERB	Verb

carefully	ADV	Adverb

Python example using NLTK:

```
None
import nltk


nltk.download('punkt_tab')

nltk.download('averaged_perceptron_tagger_eng')


sentence = "The smart student studies carefully."


tokens = nltk.word_tokenize(sentence)


result = nltk.pos_tag(tokens)


print(result)
```

Interview answer:

"POS tagging is the process of assigning grammatical labels to words in a sentence, such as nouns, verbs, adjectives and adverbs. It helps NLP systems understand the grammatical role of each word."

34. What are nouns, verbs, adjectives, adverbs and conjunctions in the context of POS tagging?

These are common grammatical categories identified during POS tagging.

Part of speech	Meaning	Example
Noun	Names a person, place or thing	John, school, book
Verb	Describes an action or state	Run, eat, is
Adjective	Describes a noun	Beautiful, intelligent
Adverb	Describes a verb, adjective or another adverb	Quickly, very
Conjunction	Connects words or sentences	And, but, because

Example sentence:

```
"John quickly reads an interesting book, but Mary sleeps."
```

- John – Noun
- Quickly – Adverb
- Reads – Verb
- Interesting – Adjective
- But – Conjunction
- Mary – Noun
- Sleeps – Verb

Interview answer:

"Nouns represent people, places or things. Verbs describe actions or states. Adjectives describe nouns, adverbs modify verbs or other descriptions, and conjunctions connect words or sentences."

35. What is syntactic parsing, and how does it differ from POS tagging?

Both techniques analyse sentence structure, but they perform different tasks.

POS tagging: Identifies the grammatical category of individual words.

Syntactic parsing: Analyses how words are grammatically connected to form a sentence.

Example:

Sentence:

`"The boy eats an apple."`

POS tagging:

- The → Determiner
- Boy → Noun
- Eats → Verb
- An → Determiner
- Apple → Noun

Syntactic parsing:

- "The boy" → Subject
- "eats" → Main verb
- "an apple" → Object

Comparison:

POS tagging	Syntactic parsing
Labels individual words	Analyses sentence structure
Identifies grammatical categories	Identifies grammatical relationships
Word-level analysis	Sentence-level analysis

Interview answer:

"POS tagging identifies the grammatical category of each word, whereas syntactic parsing analyses how words are organised and connected within a sentence."

You've hit the Free plan limit for image generations requests. You can create more images when the limit resets in 22 hours.

I can continue with the explanations and text-based diagrams for the remaining questions.

36. What is a constituency parse tree?

A constituency parse tree represents how a sentence is divided into grammatical groups called constituents.

These groups include:

- Noun Phrase (NP)
- Verb Phrase (VP)
- Prepositional Phrase (PP)

Example:

Sentence:

```
The boy eats an apple.
```

Its structure:

```
None
              S

            /   \

          NP     VP

         / \    /  \

       Det Noun V    NP

        |   |   |   / \

       The boy eats Det Noun

                    |   |
```

```
                    an apple
```

Here:

- S = Sentence
- NP = Noun Phrase
- VP = Verb Phrase
- Det = Determiner
- Noun = Naming word
- V = Verb

Interview answer:

"A constituency parse tree represents the grammatical structure of a sentence by dividing it into groups such as noun phrases and verb phrases. It shows how words combine to form larger grammatical units."

37. What is dependency parsing, and how does it represent relationships between words?

Dependency parsing identifies grammatical relationships between individual words in a sentence.

It shows which word depends on another word.

Example:

Sentence:

`"The boy eats an apple."`

Relationships:

- eats → Root (main verb)
- boy → Subject of eats
- The → Determiner of boy
- apple → Object of eats
- an → Determiner of apple

Simple representation:

```
None
          eats

         /    \

       boy    apple

        |       |

       The      an
```

Interview answer:

"Dependency parsing identifies grammatical relationships between words in a sentence. It connects words using dependency labels such as subject, object and modifier, with the main verb often acting as the root."

38. What is Named Entity Recognition (NER), and which types of entities can it identify?

Named Entity Recognition (NER) is an NLP technique used to identify and classify important entities in text.

It recognises names of people, organisations, locations, dates and other specific information.

Example:

Sentence:

```
"Johnson works at Google in India."
```

Entity	Category
Johnson	PERSON
Google	ORGANISATION

India LOCATION

Common entity types:

1. PERSON – Names of people.
2. ORGANISATION – Companies and institutions.
3. LOCATION – Countries, cities and places.
4. DATE – Dates.
5. TIME – Time expressions.
6. MONEY – Monetary values.
7. PERCENT – Percentages.

Python example using spaCy:

```
None
import spacy

nlp = spacy.load("en_core_web_sm")

text = "Johnson works at Google in India."

doc = nlp(text)

for entity in doc.ents:

    print(entity.text, entity.label_)
```

Interview answer:

"Named Entity Recognition is an NLP technique used to identify and classify important information in text, such as people, organisations, locations, dates, money and percentages."

39. What is coreference resolution, and why is it important for understanding documents?

Coreference resolution identifies different words or expressions that refer to the same person, object or entity.

It is commonly used to understand pronouns such as he, she, it and they.

Example:

```
"John bought a laptop. He uses it every day."
```

Here:

- John → He
- A laptop → It

Both references point to previously mentioned entities.

Why is it important?

- Helps understand the full meaning of documents.
- Connects pronouns with the correct entities.
- Improves information extraction.
- Helps summarisation and question answering.

Interview answer:

"Coreference resolution identifies different expressions that refer to the same entity. For example, in 'John bought a laptop. He uses it every day', it identifies that 'He' refers to John and 'it' refers to the laptop."

40. What is word sense disambiguation? Explain how a word can have different meanings depending on context.

Word Sense Disambiguation (WSD) is the process of identifying the correct meaning of a word based on its surrounding context.

The same word can have different meanings in different sentences.

Example: The word "Bank"



Sentence 1:

`"I deposited money in the bank."`

Meaning: Financial institution.

Sentence 2:

`"We sat near the bank of the river."`

Meaning: Land beside a river.

Why is WSD important?

- Helps understand sentence meaning.
- Reduces ambiguity.
- Improves machine translation.
- Supports search engines and chatbots.

Interview answer:

"Word Sense Disambiguation is the process of identifying the correct meaning of an ambiguous word using its surrounding context. For example, 'bank' can mean a financial institution or the side of a river depending on the sentence."